Scaling Layer 2 Switching 3x

4K VLANs and 64K MAC entries through optimized C data structures

Project Explanation: Summary, Components, Workflow and Architecture

Written in simple terms

> **The resume line this document explains**
> Scaled Layer 2 feature capacity 3x (4K VLANs, 64K MAC entries) by optimizing C data structures (radix trees, hash tables, memory pools) and presenting performance designs in reviews with architects across distributed teams.

Contents

1. Overall Summary (In Simple Terms) 3
1.1 Key numbers at a glance 3
2. Background Concepts You Need First 5
2.1 What is Layer 2? 5
2.2 MAC address 5
2.3 VLAN and the 4K limit 5
2.4 MAC learning, forwarding, flooding and aging 5
2.5 Control plane vs data plane 5
2.6 Big-O in one line 6
3. Components Explained (End to End) 7
3.1 Management Plane 7
3.2 Control Plane (the C software that was optimized) 8
3.3 Driver Layer 12
3.4 Data Plane (Hardware) 12
3.5 Validation and Review 13
4. Workflow Explained (End to End) 15
4.1 Master step list (A to N) 15
4.2 Workflow 1 – Creating a VLAN and adding ports 16
4.3 Workflow 2 – Learning a new MAC address (the most frequent path) 16
4.4 Workflow 3 – A device moves (MAC move) 17
4.5 Workflow 4 – Aging out a silent device 17
4.6 Workflow 5 – Port goes down (bulk flush) 17
4.7 Workflow 6 – "show mac address-table" with 64K entries 17
4.8 Workflow 7 – The engineering workflow (how the 3x was achieved) 18
5. Architecture Diagram 19
6. Architecture Diagram Explained (End to End) 20
6.1 How to read the diagram 20
6.2 Zone by zone 20
6.3 Every arrow in the diagram 21
6.4 Why the architecture achieves 3x 22
7. Glossary 23

(If page numbers are not shown, right-click the table above in Word and choose Update Field.)

1. Overall Summary (In Simple Terms)

A **network switch** is the box that connects computers, servers and phones inside a building or data center. Its job is simple: when a message (called a **frame**) arrives, it must decide which cable to send it out on. To do that quickly, the switch keeps two important lists in memory:

- **The VLAN list** – a VLAN (Virtual LAN) splits one physical switch into many separate, private networks. For example, VLAN 10 for Finance and VLAN 20 for Guests. Traffic in one VLAN cannot see the other.
- **The MAC table** – every device has a unique hardware address called a **MAC address** (like a serial number). The MAC table remembers "device X lives on port 7 in VLAN 10" so the switch can deliver frames directly instead of shouting them out of every port.

The problem: the switch software (written in C) could only handle about one-third of the VLANs and MAC addresses that customers and newer hardware needed. As the lists grew, the software slowed down, used too much memory and took too long to recover after changes.

What was done: the way the software stores and searches these lists was redesigned. Three classic computer-science tools were used:

- **Hash tables** – find any MAC address almost instantly, no matter how many are stored.
- **Radix trees** – keep entries in sorted order so the switch can quickly handle groups of entries (for example, "delete every MAC learned on port 5").
- **Memory pools** – grab memory in pre-cut chunks up front, instead of asking the operating system for memory every time, which is slow and wastes space.

The result: the switch now supports **4,096 VLANs (the maximum the Ethernet standard allows)** and **65,536 MAC entries** – about **3 times** the previous capacity – while staying fast and stable.

Teamwork part: the designs were written up with performance numbers (before vs after) and presented in **design reviews** to senior architects in teams spread across different locations, who checked and approved the approach before and after it was built.

> **One-sentence version**
> I made the switch remember three times more devices and networks by changing how its C code stores and looks up that information, and I proved the design to architects across several teams.

1.1 Key numbers at a glance

Item	Before (approx.)	After
VLANs supported	~1.3K (about one-third)	4,096 (4K) – full 12-bit VLAN ID range
MAC entries supported	~21K (about one-third)	65,536 (64K)
MAC lookup cost	Grew as table grew (list / linear search)	Constant on average (hash table)
Bulk flush (per port / VLAN)	Scan the whole table	Walk only the matching range (radix tree)
Memory allocation	malloc/free per entry (slow, fragmented)	Pre-allocated pools (fast, predictable)

Note: the "before" values are shown as approximate one-third figures implied by the 3x improvement; exact baseline numbers depend on the specific product.

2. Background Concepts You Need First

2.1 What is Layer 2?

Networks are described in layers. **Layer 2 (the Data Link layer)** is the level where switches work. They move frames between devices on the same local network using MAC addresses. (Layer 3 is routing between networks using IP addresses – not part of this project.)

2.2 MAC address

A 48-bit (6-byte) number burned into every network card, written like **00:1A:2B:3C:4D:5E**. Think of it as a device's permanent name tag.

2.3 VLAN and the 4K limit

A VLAN ID is a **12-bit number** inside the frame's 802.1Q tag. 12 bits gives 2^12 = **4,096** possible values (0 to 4095; 0 and 4095 are reserved, so 1–4094 are usable). That is why "4K VLANs" means supporting the full standard range.

2.4 MAC learning, forwarding, flooding and aging

- **Learning:** when a frame arrives on port 3 from MAC A in VLAN 10, the switch writes down "A, VLAN 10 → port 3".
- **Forwarding:** when a frame is sent to MAC A later, the switch looks it up and sends it only out of port 3.
- **Flooding:** if the destination is unknown, the switch sends the frame out of every port in that VLAN (like shouting in a hallway).
- **Aging:** if MAC A is silent for a set time (commonly 300 seconds), its entry is erased so the table does not fill with stale devices.
- **MAC move:** if MAC A suddenly appears on port 9, the entry is updated (the device was unplugged and moved).
- **Flush:** when a port goes down or a VLAN is deleted, all related MAC entries must be erased at once.

2.5 Control plane vs data plane

The **data plane** is the switch chip (ASIC) that forwards millions of frames per second in hardware. The **control plane** is the C software on the switch's CPU that decides what goes into the chip's tables. This project optimized the **control plane software**, which keeps a full copy of the tables and must keep up with the chip.

2.6 Big-O in one line

O(1) means "takes the same time no matter how big the table is". **O(n)** means "twice as many entries = twice as slow". **O(log n)** or **O(key length)** is in between and grows very slowly. Moving the hot paths from O(n) to O(1) is what allowed 3x more entries without slowing down.

3. Components Explained (End to End)

The system has 17 components grouped into five areas. The numbers in brackets match the numbers in the architecture diagram (Section 5).

Area	Components	Purpose
Management plane	[1] Operator Interfaces, [2] Configuration Manager	How humans and tools give orders to the switch
Control plane	[3] L2 Manager, [4] VLAN Manager, [5] MAC Learning Engine, [6] Aging Engine, [7] VLAN Table, [8] MAC Hash Table, [9] Radix Tree Index, [10] Memory Pools	The C software brain – this is where the optimization happened
Driver layer	[11] Hardware Abstraction Layer (HAL / SDK)	Translator between software and the chip
Data plane	[12] ASIC VLAN Table, [13] ASIC MAC (FDB) Table, [14] Switch Ports / Packets	The hardware that actually moves frames
Validation and review	[15] Scale Test Harness, [16] Profiling & Metrics, [17] Design Reviews	Proving the 3x gain and getting architect approval

3.1 Management Plane

[1] Operator Interfaces

Where it lives: Top of the stack, facing the user.

In simple terms: The ways a network engineer or automation tool talks to the switch: typing commands, sending API requests, or polling statistics.

Everyday analogy: *The front desk of a hotel where guests make requests.*

Minute details:

- **CLI** (command-line interface), e.g. "vlan 100", "switchport access vlan 100", "show mac address-table".
- **NETCONF / REST APIs** let automation scripts push hundreds of VLANs at once – this is where 4K-VLAN bulk configuration actually comes from.
- **SNMP** lets monitoring systems read counters such as "how many MACs are learned".
- "show" commands at 64K entries must print in sorted, page-by-page form without freezing the CPU – this depends on the radix tree [9].

[2] Configuration Manager

Where it lives: Management plane, between the interfaces and the L2 Manager.

In simple terms: Checks every request is valid and safe before passing it on.

Everyday analogy: *A receptionist who checks your booking is real before giving you a key.*

Minute details:

- Validates ranges: VLAN ID must be 1–4094; port must exist; static MAC must be a valid unicast address.
- Checks limits: refuses a new entry when the 64K MAC or 4K VLAN capacity is reached, returning a clear error instead of crashing.
- Stores the saved configuration so the switch can rebuild everything after a reboot.
- Converts the request into a single, typed call to the L2 Manager (step B in the diagram).

3.2 Control Plane (the C software that was optimized)

[3] L2 Manager (central API)

Where it lives: Top of the control plane.

In simple terms: The one doorway into all Layer 2 logic. Every VLAN or MAC change, whether it comes from a user or from hardware, passes through here.

Everyday analogy: *A traffic police officer at a junction sending each car down the right road.*

Minute details:

- Exposes functions such as l2_vlan_create(), l2_vlan_add_port(), l2_mac_add(), l2_mac_flush_port().
- Routes VLAN work to the VLAN Manager [4] (step C) and MAC work to the MAC Learning Engine [5] (step D).
- Receives learn and age events coming up from hardware (step M) and dispatches them.
- Sends final changes down to the HAL [11] (step I) so the chip stays in sync.
- Uses locking so two threads never change the same entry at the same time.

What was optimized for the 3x scale:

- Grouped many small hardware updates into batches so that a 4K-VLAN configuration push does not become 4K separate slow chip writes.

[4] VLAN Manager

Where it lives: Control plane, engine row.

In simple terms: Creates and deletes VLANs and decides which ports belong to which VLAN.

Everyday analogy: *A building manager deciding which rooms belong to which department.*

Minute details:

- Handles create / delete VLAN, add / remove port, and whether a port is **tagged** (trunk, carries many VLANs) or **untagged** (access, one VLAN).
- On VLAN delete, asks the MAC engine to flush every MAC in that VLAN.
- Writes results into the VLAN Table [7] (step E).

[5] MAC Learning Engine

Where it lives: Control plane, engine row.

In simple terms: Adds, updates and removes MAC address entries.

Everyday analogy: *A post office clerk updating the address book when people move house.*

Minute details:

- **New MAC:** allocate an entry from the pool [10], insert into the hash table [8] (step F) and radix tree [9] (step G).
- **Known MAC, same port:** just refresh the "last seen" time.
- **Known MAC, new port (MAC move):** update the port; count moves to catch loops (a MAC flapping between ports).
- **Static MACs:** configured by the user, never aged out.
- **Flush:** remove all entries for a port or VLAN using a radix-tree range walk instead of scanning all 64K entries.
- **Rate limiting:** caps how many learns per second are processed so a burst cannot starve the CPU.

What was optimized for the 3x scale:

- Learning used to search a list; it now does one hash lookup, so learn rate stays high even with 64K entries.

[6] Aging Engine

Where it lives: Control plane, engine row.

In simple terms: Removes MAC entries for devices that have gone quiet.

Everyday analogy: *A librarian removing names of members who have not visited in months.*

Minute details:

- Default age time is commonly 300 seconds (configurable).
- Uses a **timer wheel**: time is split into slots like a clock face; each entry sits in the slot for when it should expire. Every tick, only one slot is checked – not all 64K entries.
- Can also read the hardware "hit bit" (the chip marks entries that saw traffic) to decide whether an entry is still alive.

- Deletes expired entries from the hash table, radix tree and hardware, and returns the memory to the pool (step H).

What was optimized for the 3x scale:

- Replaced a full-table scan every few seconds (very costly at 64K) with the timer wheel plus ordered radix-tree sweeps, spreading the work evenly.

[7] VLAN Table

Where it lives: Control plane, data-structure row.

In simple terms: The software list of all VLANs and their member ports.

Everyday analogy: *A row of 4,096 numbered mailboxes – to find VLAN 100 you go straight to box 100.*

Minute details:

- A **direct-index array** of 4,096 slots: the VLAN ID is the array index, so lookup is O(1) with no searching.
- Each slot holds a **port bitmap**: one bit per port (1 = member). Checking or changing membership is a single bit operation.
- Separate bitmaps for tagged and untagged members.
- Kept compact (only small fixed fields per slot, details allocated from the pool when the VLAN actually exists) so 4K slots use little memory.

What was optimized for the 3x scale:

- Replaced a linked list of VLANs (search time grew with every VLAN) by the fixed array, making all 4K VLANs equally fast.

[8] MAC Hash Table

Where it lives: Control plane, data-structure row.

In simple terms: The main store for up to 64K MAC entries, built for instant lookup.

Everyday analogy: *A coat-check: your ticket number tells the attendant exactly which hook to go to.*

Minute details:

- **Key** = MAC address (6 bytes) + VLAN ID (2 bytes). The same MAC can exist in two VLANs, so both are needed.
- A **hash function** mixes the key into a bucket number. A fast, well-mixing hash (e.g. CRC- or Jenkins-style) spreads entries evenly so buckets stay short.
- **Collisions** (two keys landing in the same bucket) are handled by chaining – a short list inside the bucket.
- Bucket count sized as a power of two above 64K so the **load factor** stays low and average chain length is about 1.

- **Entry fields:** MAC, VLAN, port, type (dynamic/static), last-seen time, flags, and links for the hash chain and the radix tree.
- Lookup, insert and delete are **O(1) on average**.

What was optimized for the 3x scale:

- Entry structure was packed tightly (fields ordered to avoid padding) so more entries fit per cache line and total memory for 64K entries stayed small.
- Moving from linear search to hashing is the single biggest reason 3x more entries did not mean 3x slower learning.

[9] Radix Tree Index

Where it lives: Control plane, data-structure row.

In simple terms: A second, sorted index over the same MAC entries, used when we need groups or order rather than one entry.

Everyday analogy: *A phone book sorted by area code: to find everyone in area 555 you open one section, not the whole book.*

Minute details:

- A **radix tree (trie)** stores keys bit-by-bit or byte-by-byte; entries that share a prefix share a branch.
- Keys are arranged as VLAN + MAC (or port + MAC), so all entries for one VLAN or one port sit together in one sub-tree.
- **Range walk:** flushing VLAN 100 visits only VLAN 100's branch instead of all 64K entries.
- **Sorted output:** "show mac address-table" can print in order and resume page-by-page from the last key.
- Search time depends on key length (fixed 8 bytes), not on the number of entries – so it does not slow down as the table grows.
- Path compression (skipping single-child nodes) keeps the tree small.

What was optimized for the 3x scale:

- Port-down and VLAN-delete flushes went from full-table scans to targeted walks – critical for fast network recovery at 64K entries.

[10] Memory Pools

Where it lives: Control plane, under all tables.

In simple terms: A big block of memory cut into equal-size pieces up front, handed out and taken back very quickly.

Everyday analogy: *An ice-cube tray: all cubes are the same size and ready to grab instantly.*

Minute details:

- At startup, memory for 64K MAC entries (and radix nodes, VLAN details) is reserved in one go.
- Free pieces are kept on a **free list**; allocate = take the first piece, free = put it back. Both are O(1).
- No calls to malloc()/free() on the hot path, which are slow, need locks and cause **fragmentation** (memory full of small unusable gaps).
- Fixed capacity gives a clear, predictable limit: when the pool is empty, the table is full and a clean error is returned.
- Pool statistics (in use, high-water mark) help debugging and capacity reports.
- Same-size neighbours in memory improve CPU cache behaviour.

What was optimized for the 3x scale:

- Removed allocator overhead from learning and aging and eliminated fragmentation that previously limited how many entries could be stored.

3.3 Driver Layer

[11] Hardware Abstraction Layer (HAL / SDK)

Where it lives: Between control-plane software and the switch chip.

In simple terms: Translates software changes into the chip's own language and reports hardware events back up.

Everyday analogy: *An interpreter between a manager who speaks English and a factory that speaks only machine code.*

Minute details:

- Uses the chip vendor's SDK to write VLAN entries (step J) and MAC entries (step K) into hardware tables.
- **Batching / DMA:** sends many updates in one transfer, which matters when installing thousands of entries.
- Receives **learn events** from the chip (new source MAC seen) and **age / hit-bit** information (step L) and sends them up to the L2 Manager (step M).
- Hides chip differences so the same C code works on several hardware platforms.

3.4 Data Plane (Hardware)

[12] ASIC VLAN Table

Where it lives: Inside the switch chip.

In simple terms: The hardware copy of VLAN membership used for every frame.

Everyday analogy: *The printed floor map at a building entrance that guards actually use.*

Minute details:

- Programmed by the HAL from the software VLAN Table [7].
- Used per frame to check a port is allowed in a VLAN and which ports to flood to.

[13] ASIC MAC (FDB) Table

Where it lives: Inside the switch chip.

In simple terms: The hardware copy of MAC → port entries (FDB = Forwarding Database).

Everyday analogy: *The sorting machine at a post office that reads an address and drops the letter in the right bin in an instant.*

Minute details:

- Hardware has its own hash-based table; the software copy must match it exactly.
- Supporting 64K in hardware is pointless unless software can also track 64K – this project made software match the hardware capacity.

[14] Switch Ports / Packets

Where it lives: The physical ports.

In simple terms: Where frames enter and leave the switch.

Everyday analogy: *The doors of the building.*

Minute details:

- Known destination MAC → frame goes out one port.
- Unknown destination → frame is flooded within the VLAN.
- Unknown source MAC → chip raises a learn event (step L) so software can add the entry.

3.5 Validation and Review

[15] Scale Test Harness

Where it lives: Lab test setup.

In simple terms: Test equipment and scripts that push the switch to its limits.

Everyday analogy: *A stress test on a bridge – drive heavy trucks over it before opening to the public.*

Minute details:

- Traffic generators send frames from 64K different source MACs across 4K VLANs.
- Measures **learn rate** (MACs/second), **memory use**, **CPU load**, **flush time** and **convergence time** (how fast the switch recovers after a port goes down).
- Repeats tests at 25%, 50%, 75% and 100% of capacity to confirm speed stays flat as tables fill.

[16] Profiling & Metrics

Where it lives: Lab and development machines.

In simple terms: Tools that show exactly where time and memory are spent in the C code.

Everyday analogy: *A fitness tracker that shows which part of your run is slowest.*

Minute details:

- Profilers such as perf / gprof find hot functions (e.g. the old linear search).
- Counters for cache misses, memory footprint per entry and pool high-water marks.
- **Before vs after** numbers are the evidence that the change delivered the 3x gain (step N feeds this back into the code).

[17] Design Reviews

Where it lives: Meetings with architects across distributed (multi-site) teams.

In simple terms: Presenting the design and the data so senior engineers can challenge, improve and approve it.

Everyday analogy: *An architect showing building plans to the city inspectors before construction.*

Minute details:

- A written design document: problem, goals (4K/64K), options considered (e.g. balanced tree vs radix tree vs hash), chosen design, memory math and risks.
- Performance graphs from [15] and [16] back up every claim.
- Reviewers in other teams (hardware, platform, protocols such as Spanning Tree) check their features are not broken.
- Feedback is applied to the code and the design is re-presented until approved.
- Working across time zones means clear written documents and recorded decisions.

4. Workflow Explained (End to End)

This section walks through how the system works step by step. The letters A–N match the red circles on the architecture diagram.

4.1 Master step list (A to N)

Step	From → To	What happens (simple terms)
A	Operator Interfaces → Configuration Manager	A user or script sends a command, e.g. create VLAN 100.
B	Configuration Manager → L2 Manager	The checked, valid request is handed to the central L2 API.
C	L2 Manager → VLAN Manager	VLAN-related work is sent to the VLAN Manager.
D	L2 Manager → MAC Learning Engine	MAC-related work (learn, move, static add, flush) is sent to the MAC engine.
E	VLAN Manager → VLAN Table	The VLAN slot and port bitmap are updated.
F	MAC Learning Engine → MAC Hash Table	Entry inserted / updated / deleted for O(1) lookup.
G	MAC Learning Engine → Radix Tree Index	Same entry linked into the sorted index for range work.
H	Aging Engine → Radix Tree Index	Aging sweeps walk ordered entries and remove expired ones (also from the hash table).
(pool)	Memory Pools → all tables	Entries and nodes are taken from / returned to pre-allocated pools.
I	L2 Manager → HAL	Final changes are batched and sent toward the hardware.
J	HAL → ASIC VLAN Table	VLAN membership written into the chip.
K	HAL → ASIC MAC Table	MAC entries written into the chip.
L	Switch Ports → HAL	Chip reports a new source MAC (learn event) or aging / hit info.
M	HAL → L2 Manager	Events are delivered up into the software.
N	Profiling & Metrics → Control Plane	Measurements feed back into code changes and design reviews.

4.2 Workflow 1 – Creating a VLAN and adding ports

Example: an engineer types "vlan 100" and then puts ports 1–4 into it.

1. **(A)** The CLI reads the command and sends it to the Configuration Manager.
2. The Configuration Manager checks that 100 is between 1 and 4094 and that the 4K VLAN limit is not reached.
3. **(B)** It calls the L2 Manager with "create VLAN 100".
4. **(C)** The L2 Manager routes the call to the VLAN Manager.
5. **(E)** The VLAN Manager goes straight to slot 100 of the direct-index array (no search) and marks it active; extra details come from the memory pool.
6. For "add ports 1–4", it sets bits 1–4 in the port bitmap – one quick bit operation per port.
7. **(I)** The L2 Manager sends the change to the HAL, batching it with other pending changes.
8. **(J)** The HAL programs the chip's VLAN table. Hardware can now forward VLAN 100 traffic.
9. The command returns "success" to the user. With 4K VLANs pushed by a script, each one takes the same short time.

4.3 Workflow 2 – Learning a new MAC address (the most frequent path)

Example: a laptop with MAC A sends its first frame on port 3, VLAN 100.

1. The frame enters port 3. The chip looks up MAC A in its MAC table and does not find it.
2. The chip still forwards (or floods) the frame so the user sees no delay.
3. **(L)** The chip raises a learn event "MAC A, VLAN 100, port 3" to the HAL.
4. **(M)** The HAL passes the event to the L2 Manager, which hands it to the MAC Learning Engine **(D)**.
5. The engine hashes (MAC A + VLAN 100) and checks one bucket of the hash table – **O(1)**.
6. Not found, so it takes a free entry from the **memory pool** (O(1), no malloc).
7. **(F)** It fills the entry (MAC, VLAN, port 3, dynamic, last-seen = now) and inserts it into the hash bucket.
8. **(G)** It links the same entry into the radix tree under VLAN 100 / port 3.
9. The entry is placed into the Aging Engine's timer wheel slot for "now + 300 s".
10. **(I, K)** The L2 Manager / HAL writes the entry into the chip's MAC table.
11. Next time someone sends to MAC A, the chip forwards it straight to port 3 – no flooding.

4.4 Workflow 3 – A device moves (MAC move)

Example: the laptop is unplugged from port 3 and plugged into port 9.

1. A frame from MAC A arrives on port 9. The chip sees the port differs from its table and raises an event **(L, M)**.
2. The MAC engine finds MAC A with one hash lookup.
3. It updates the port to 9 and re-links the entry in the radix tree under port 9 (removed from port 3's branch).
4. A move counter is increased; too many moves in a short time signals a possible network loop and raises an alarm.
5. The hardware entry is updated **(K)**. Traffic now goes to port 9.

4.5 Workflow 4 – Aging out a silent device

Example: a printer has not sent anything for over 5 minutes.

1. The timer wheel ticks (e.g. every second) and checks only the current slot – not all 64K entries.
2. For each entry in that slot, the engine checks the last-seen time and the hardware hit bit.
3. If traffic was seen recently, the entry is moved to a later slot (kept alive).
4. If not, **(H)** the entry is removed from the radix tree and the hash table.
5. The entry's memory goes back to the pool free list in O(1).
6. The HAL deletes it from the chip's MAC table. The table stays clean without big CPU spikes.

4.6 Workflow 5 – Port goes down (bulk flush)

Example: port 5 loses its link and had 2,000 MACs learned on it.

1. The hardware reports link down; the L2 Manager asks the MAC engine to flush port 5.
2. The engine jumps to port 5's branch in the radix tree and walks only those 2,000 entries – not all 64K.
3. Each entry is unlinked from the hash table and radix tree and returned to the pool.
4. Deletes are sent to the hardware in batches by the HAL.
5. Traffic for those devices is flooded until they are re-learned on their new path, so the network converges quickly.

4.7 Workflow 6 – "show mac address-table" with 64K entries

1. The operator runs the show command **(A, B)**.
2. The L2 Manager walks the radix tree in sorted order and returns one page of entries.
3. The last key printed is remembered; the next page resumes from that key.
4. The CPU never freezes building one giant list, and output is neatly sorted by VLAN and MAC.

4.8 Workflow 7 – The engineering workflow (how the 3x was achieved)

This is the project process itself, from problem to delivery.

1. **Measure the baseline:** use the scale harness [15] to fill tables until learning slowed or memory ran out; record the limits (roughly one-third of target).
2. **Profile:** use perf / gprof [16] to find hot spots – linear searches, full-table aging scans, and malloc/free overhead.
3. **Design:** choose a direct-index array for VLANs, a hash table for MAC lookups, a radix tree for ordered/range operations, and memory pools for allocation; calculate memory per entry for 64K entries.
4. **Design review (round 1):** present the plan and memory math to architects in distributed teams; collect feedback (e.g. locking, hardware-sync, effect on other features).
5. **Implement in C:** build the structures, pack entry layouts, add batching to the HAL, add unit tests for insert / delete / move / flush / age.
6. **Validate at scale:** rerun the harness at 25–100% load; confirm flat learn rate, bounded memory and fast flush / convergence.
7. **Design review (round 2):** present before-vs-after data proving the 3x gain; get sign-off.
8. **Release:** the feature ships with new published limits of 4K VLANs and 64K MAC entries.

5. Architecture Diagram

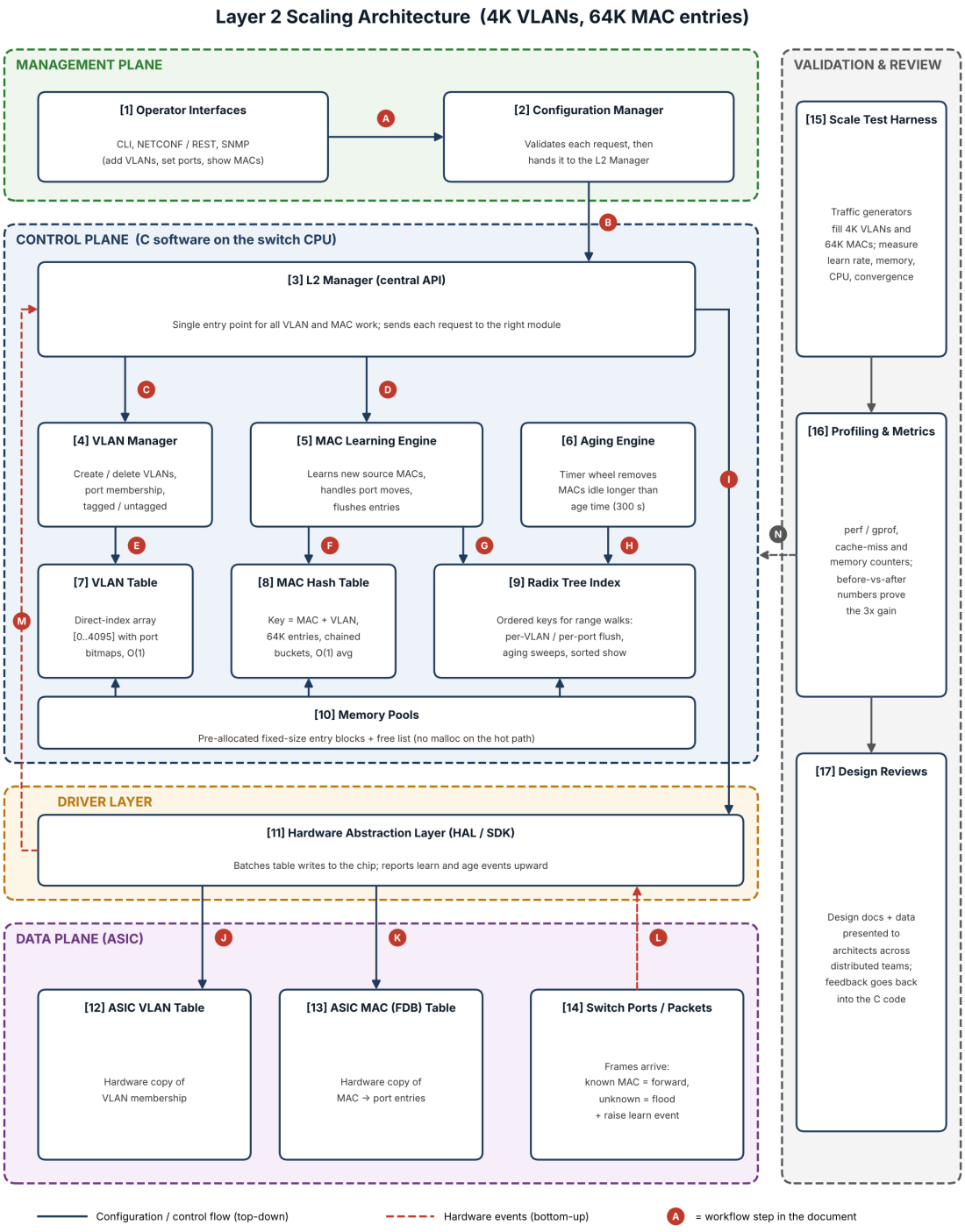


Figure 1 – Layer 2 scaling architecture. Boxes are numbered [1]–[17]; red circles A–N are workflow steps.

6. Architecture Diagram Explained (End to End)

6.1 How to read the diagram

- **Dashed coloured frames** are zones (layers). Top to bottom: Management → Control → Driver → Data. The grey column on the right is Validation & Review.
- **White boxes** are components. The number in brackets matches Section 3.
- **Solid dark-blue arrows** show configuration and control flowing **downward** (from people to hardware).
- **Dashed red arrows** show hardware events flowing **upward** (from ports to software).
- **Grey arrows** on the right show the validation and review loop.
- **Red circles A–N** are workflow steps described in Section 4.

6.2 Zone by zone

Management Plane (green, top)

Contains [1] Operator Interfaces and [2] Configuration Manager side by side. Arrow **A** goes left to right: commands enter at [1] and are checked in [2]. Arrow **B** drops from [2] down into the Control Plane.

Control Plane (blue, middle – the heart of the project)

Organised in four rows so the hierarchy is clear:

- **Row 1 – [3] L2 Manager** spans the full width, showing it is the single entry point for everything below.
- **Row 2 – engines:** [4] VLAN Manager, [5] MAC Learning Engine, [6] Aging Engine. Arrows **C** and **D** come down from the L2 Manager. (The Aging Engine runs on its own timer, so it has no arrow from above.)
- **Row 3 – data structures:** [7] VLAN Table, [8] MAC Hash Table, [9] Radix Tree Index. Arrow **E** joins the VLAN Manager to the VLAN Table; **F** and **G** show the MAC engine writing every entry into both the hash table and the radix tree; **H** shows the Aging Engine walking the radix tree to expire entries.
- **Row 4 – [10] Memory Pools** spans the bottom; three short upward arrows show every table taking its entries from the pools. Placing it underneath shows it is the foundation that everything else sits on.
- **Right-side lane (arrow I):** a line leaves the right edge of the L2 Manager and runs straight down to the HAL – the path for all changes to reach hardware.
- **Left-side lane (arrow M, red dashed):** a line leaves the left edge of the HAL and runs up to the L2 Manager – the path for learn / age events coming up from hardware.

Driver Layer (orange)

A single wide box, [11] HAL / SDK, because every piece of hardware traffic in either direction passes through it. Arrow **I** enters from above; **M** leaves to the left going up; **J** and **K** leave downward; **L** enters from below.

Data Plane (purple, bottom)

[12] ASIC VLAN Table and [13] ASIC MAC Table receive programming from the HAL (arrows **J** and **K**). [14] Switch Ports / Packets sends learn events up to the HAL (red dashed arrow **L**). These boxes are the hardware mirrors of the software tables [7] and [8].

Validation & Review (grey column, right)

[15] Scale Test Harness → [16] Profiling & Metrics → [17] Design Reviews, connected top to bottom, show the order of the engineering loop: stress the system, measure it, then present the evidence. The dashed grey arrow **N** from [16] points into the Control Plane, meaning measurements drive changes to the C code.

6.3 Every arrow in the diagram

Arrow	Start	End	Meaning
A	[1] Operator Interfaces	[2] Configuration Manager	User command enters the system
B	[2] Configuration Manager	[3] L2 Manager	Validated request goes to the L2 API
C	[3] L2 Manager	[4] VLAN Manager	VLAN work dispatched
D	[3] L2 Manager	[5] MAC Learning Engine	MAC work and learn events dispatched
E	[4] VLAN Manager	[7] VLAN Table	Direct-index slot and bitmap update
F	[5] MAC Learning Engine	[8] MAC Hash Table	O(1) insert / update / delete
G	[5] MAC Learning Engine	[9] Radix Tree Index	Keep sorted index in sync
H	[6] Aging Engine	[9] Radix Tree Index	Ordered sweep to expire idle MACs
(3 short)	[10] Memory Pools	[7], [8], [9]	Entries come from pre-allocated pools
I	[3] L2 Manager	[11] HAL	Batched changes sent toward hardware

Arrow	Start	End	Meaning
J	[11] HAL	[12] ASIC VLAN Table	Program VLAN membership in chip
K	[11] HAL	[13] ASIC MAC Table	Program MAC entries in chip
L	[14] Switch Ports	[11] HAL	Learn / age events from hardware (upward)
M	[11] HAL	[3] L2 Manager	Events delivered to software (upward)
(grey)	[15] → [16] → [17]	—	Test, measure, review loop
N	[16] Profiling & Metrics	Control Plane	Data drives code optimization

6.4 Why the architecture achieves 3x

Bottleneck before	Fix in the architecture	Effect at 4K / 64K
VLANs stored in a list	[7] Direct-index array + port bitmaps	Every VLAN lookup O(1)
MAC lookup by linear search	[8] Hash table keyed by MAC + VLAN	Learn rate stays flat as table grows
Flush / show scanned whole table	[9] Radix tree range walks	Only touch affected entries; sorted paging
Aging scanned all entries	[6] Timer wheel + ordered sweeps	No CPU spikes; smooth background work
malloc / free per entry	[10] Memory pools with free list	O(1) alloc, no fragmentation, fixed footprint
One chip write per change	[11] Batched HAL updates	Bulk VLAN/MAC programming is fast

7. Glossary

Term	Simple meaning
ASIC	The custom switch chip that forwards frames in hardware.
Bitmap	A row of bits, one per port; 1 means "member".
Cache line	A small block (usually 64 bytes) the CPU reads at once; packing data well makes code faster.
Chaining	Handling hash collisions by keeping a short list in each bucket.
Control plane	Software on the CPU that decides what goes in the tables.
Data plane	Hardware that moves frames using those tables.
FDB	Forwarding Database – another name for the MAC table.
Flooding	Sending a frame out of all ports in a VLAN when the destination is unknown.
Fragmentation	Free memory broken into small gaps that cannot be used well.
HAL / SDK	Software layer that talks to the chip.
Hash table	Structure that finds an item in constant time by computing its bucket.
Load factor	Entries ÷ buckets; lower means shorter chains and faster lookups.
MAC address	48-bit unique hardware address of a device.
Memory pool	Pre-reserved equal-size memory pieces handed out quickly.
Radix tree (trie)	Tree that stores keys by their bits/bytes, keeping them sorted and grouped by prefix.
Timer wheel	Clock-like structure that groups items by expiry time so only one slot is checked per tick.
VLAN	Virtual LAN – a separate logical network on the same switch; IDs 1–4094.